DSA Lab Sheet – 05: Stack

Subject: Data Structure I (BTCS209P) | Language: C++ | Semester: III

1. Stack using Array — Push, Pop, Peek and Display

Algorithm: Initialize top = -1. Push checks overflow, increments top, inserts element. Pop checks underflow, displays and decrements top. Peek displays stack[top]. Display prints elements from top to 0.

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) { cout << "Stack Overflow
"; return; }
    stackArr[++top] = x;
}

void pop() {
    if (top == -1) { cout << "Stack Underflow
"; return; }
    cout << stackArr[top--] << " popped
";
}

void peek() {
    if (top == -1) { cout << "Stack is Empty
"; return; }
    cout << "Top = " << stackArr[top] << endl;
}

void display() {
    if (top == -1) { cout << "Stack is Empty
"; return; }
    for (int i = top; i >= 0; i--) cout << stackArr[i] << " ";
    cout << endl;
}

int main() {
    push(10); push(20); push(30);
    display(); peek(); pop(); display();
    return 0;
}
```

Time: Push O(1), Pop O(1), Peek O(1), Display O(n) | Space: O(n)

2. Demonstrate Stack Overflow

```
#include <iostream>
using namespace std;

#define MAX 3
int stackArr[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) { cout << "Stack Overflow
"; return; }
    stackArr[++top] = x;
    cout << x << " pushed
";
}

int main() {
    push(10); push(20); push(30);
    push(40);   // Overflow
    return 0;
}
```

Output: 10 pushed | 20 pushed | 30 pushed | Stack Overflow | Time: O(1) | Space: O(n)

3. Demonstrate Stack Underflow

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void pop() {
    if (top == -1) { cout << "Stack Underflow
"; return; }
    cout << stackArr[top--] << " popped
";
}

int main() {
    pop();
    return 0;
}
```

Output: Stack Underflow | Time: O(1) | Space: O(n)

4. Display TOP Element Using Peek

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) cout << "Stack Overflow
";
    else stackArr[++top] = x;
}

void peek() {
    if (top == -1) cout << "Stack is Empty
";
    else cout << "Top element = " << stackArr[top] << endl;
}

int main() {
    push(10); push(20); push(30);
    peek();
    return 0;
}
```

Output: Top element = 30 | Time: O(1) | Space: O(n)

5. Menu-Driven Stack

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void push() {
    int x; cout << "Enter value: "; cin >> x;
    if (top == MAX - 1) cout << "Stack Overflow
";
    else stackArr[++top] = x;
}

void pop() {
    if (top == -1) cout << "Stack Underflow
";
    else cout << stackArr[top--] << " popped
";
}

void peek() {
    if (top == -1) cout << "Stack is Empty
";
```

```
    else cout << "Top = " << stackArr[top] << endl;
}

void display() {
    if (top == -1) { cout << "Stack is Empty
"; return; }
    for (int i = top; i >= 0; i--) cout << stackArr[i] << " ";
    cout << endl;
}

int main() {
    int choice;
    do {
        cout << "
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: ";
        cin >> choice;
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: cout << "Exit"; break;
            default: cout << "Invalid choice
";
        }
    } while (choice != 5);
    return 0;
}
```

Time: Push/Pop/Peek O(1), Display O(n) | Space: O(n)

6. Stack of Integers Using Array

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) cout << "Overflow
";
    else stackArr[++top] = x;
}

void display() {
    for (int i = top; i >= 0; i--) cout << stackArr[i] << " ";
}

int main() {
    push(5); push(10); push(15); push(20);
    cout << "Integer Stack: ";
    display();
    return 0;
}
```

Time: Push O(1), Display O(n) | Space: O(n)

7. Reverse a String Using Stack

Algorithm: Read string, push characters into stack, pop them one by one to get the reversed string.

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100
char stackArr[MAX];
int top = -1;

int main() {
    string str;
    cout << "Enter string: "; cin >> str;
```

```
    for (char ch : str) stackArr[++top] = ch;
    cout << "Reversed string: ";
    while (top != -1) cout << stackArr[top--];
    return 0;
}
```

Input: HELLO | Output: OLLEH | Time: O(n) | Space: O(n)

8. Check Balanced Parentheses

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100
char stackArr[MAX];
int top = -1;

bool matching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

int main() {
    string exp;
    cout << "Enter expression: "; cin >> exp;
    for (char ch : exp) {
        if (ch == '(' || ch == '[' || ch == '{') stackArr[++top] = ch;
        else if (ch == ')' || ch == ']' || ch == '}') {
            if (top == -1 || !matching(stackArr[top], ch)) {
                cout << "Not Balanced"; return 0;
            }
            top--;
        }
    }
    if (top == -1) cout << "Balanced";
    else cout << "Not Balanced";
    return 0;
}
```

Input: {[()]} | Output: Balanced | Time: O(n) | Space: O(n)

9. Infix to Postfix

Example: A+B*C → ABC*+

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

char stackArr[100];
int top = -1;

int priority(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int main() {
    string infix, postfix = "";
    cout << "Enter infix: "; cin >> infix;
    for (char ch : infix) {
        if (isalnum(ch)) postfix += ch;
        else if (ch == '(') stackArr[++top] = ch;
        else if (ch == ')') {
            while (top != -1 && stackArr[top] != '(') postfix += stackArr[top--];
            top--;
        } else {
            while (top != -1 && priority(stackArr[top]) >= priority(ch)) postfix += stackArr[top--];
            stackArr[++top] = ch;
        }
    }
    while (top != -1) postfix += stackArr[top--];
    cout << "Postfix: " << postfix;
```

```
    return 0;
}
```

Time: O(n) | Space: O(n)

10. Evaluate Postfix Expression

Example: 23*54*+ = 26

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int stackArr[100];
int top = -1;

int main() {
    string exp;
    cout << "Enter postfix: "; cin >> exp;
    for (char ch : exp) {
        if (isdigit(ch)) stackArr[++top] = ch - '0';
        else {
            int b = stackArr[top--];
            int a = stackArr[top--];
            switch (ch) {
                case '+': stackArr[++top] = a + b; break;
                case '-': stackArr[++top] = a - b; break;
                case '*': stackArr[++top] = a * b; break;
                case '/': stackArr[++top] = a / b; break;
            }
        }
    }
    cout << "Result = " << stackArr[top];
    return 0;
}
```

Input: 23*54*+ | Output: Result = 26 | Time: O(n) | Space: O(n)

11. Infix to Prefix

Example: A+B*C → +A*BC

```
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
using namespace std;

char stackArr[100];
int top = -1;

int priority(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int main() {
    string infix, prefix = "";
    cout << "Enter infix: "; cin >> infix;
    reverse(infix.begin(), infix.end());
    for (char &ch : infix) {
        if (ch == '(') ch = ')';
        else if (ch == ')') ch = '(';
    }
    for (char ch : infix) {
        if (isalnum(ch)) prefix += ch;
        else if (ch == '(') stackArr[++top] = ch;
        else if (ch == ')') {
            while (top != -1 && stackArr[top] != '(') prefix += stackArr[top--];
            top--;
        } else {
            while (top != -1 && priority(stackArr[top]) > priority(ch)) prefix += stackArr[top--];
            stackArr[++top] = ch;
        }
    }
    while (top != -1) prefix += stackArr[top--];
    reverse(prefix.begin(), prefix.end());
```

```
    cout << "Prefix: " << prefix;
    return 0;
}
```

Time: O(n) | Space: O(n)

12. Two Stacks in a Single Array

Concept: Stack 1 grows left to right, Stack 2 grows right to left. Overflow when top1 + 1 == top2.

```
#include <iostream>
using namespace std;

#define MAX 10
int arr[MAX];
int top1 = -1;
int top2 = MAX;

void push1(int x) {
    if (top1 + 1 == top2) cout << "Overflow
";
    else arr[++top1] = x;
}

void push2(int x) {
    if (top1 + 1 == top2) cout << "Overflow
";
    else arr[--top2] = x;
}

void display() {
    cout << "Stack 1: ";
    for (int i = top1; i >= 0; i--) cout << arr[i] << " ";
    cout << "
Stack 2: ";
    for (int i = top2; i < MAX; i++) cout << arr[i] << " ";
}

int main() {
    push1(10); push1(20); push2(100); push2(200);
    display();
    return 0;
}
```

Time: Push O(1) | Space: O(n)

13. Stack Using a Class

```
#include <iostream>
using namespace std;

class Stack {
    int arr[5];
    int top;
public:
    Stack() { top = -1; }
    void push(int x) {
        if (top == 4) cout << "Overflow
";
        else arr[++top] = x;
    }
    void pop() {
        if (top == -1) cout << "Underflow
";
        else cout << arr[top--] << " popped
";
    }
    void peek() {
        if (top == -1) cout << "Empty
";
        else cout << "Top = " << arr[top] << endl;
    }
    void display() {
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
    }
};

int main() {
    Stack s;
```

```
    s.push(10); s.push(20); s.push(30);
    s.display(); cout << endl;
    s.peek(); s.pop();
    return 0;
}
```

Time: Push/Pop/Peek O(1), Display O(n) | Space: O(n)

14. Stack Use in Recursion

Recursion internally uses a call stack to store function calls.

```
#include <iostream>
using namespace std;

void countdown(int n) {
    if (n == 0) return;
    cout << n << " ";
    countdown(n - 1);
}

int main() {
    cout << "Recursion using call stack:
";
    countdown(5);
    return 0;
}
```

Output: 5 4 3 2 1 | Time: O(n) | Space: O(n) (recursion stack)

15. Analyze Time Complexity of Stack Operations

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;

void push(int x) { if (top == MAX - 1) return; stackArr[++top] = x; }
void pop() { if (top == -1) return; top--; }
void peek() { if (top == -1) return; cout << stackArr[top] << endl; }
void display() { for (int i = top; i >= 0; i--) cout << stackArr[i] << " "; }

int main() {
    push(10); push(20); push(30);
    cout << "Peek: "; peek();
    cout << "Display: "; display();
    return 0;
}
```

Operation	Time Complexity	Space Complexity
Push	O(1)	O(n)
Pop	O(1)	O(n)
Peek	O(1)	O(n)
Display	O(n)	O(n)